

REST 練習 4：URI 路徑參數與單筆查詢

Lecture

rs01~rs03 的 URL 都是固定的 (`/hello`、`/carriers`、`/flights`)，沒有辦法「指定要查哪一筆」。REST 的做法是把「要查哪一筆」也編碼進 URL 路徑本身，例如 `/flights/AA/17/20260115` 代表「這一個特定的航班」——這種寫在路徑裡、隨請求變動的片段叫**路徑參數** (Path Parameter)。

這題也是「集合資源」跟「單一資源」這組 REST 常見概念的第一次示範：`/flights` (集合，回傳陣列) 跟 `/flights/{carrid}/{connid}/{fldate}` (單一資源，回傳一個物件) 是兩個不同的 URI，也理所當然是兩個不同的 Resource Class——不要想著在同一個 GET 方法裡用 if/else 判斷「這次是查集合還是查單筆」，REST 的做法是讓 URI 本身就決定要交給誰處理。

路徑參數的設計原則：URI 要能唯一定位到一筆資源，也就是要包含足夠的欄位涵蓋資料表的主鍵 (這題花了不少篇幅解釋為什麼要三個欄位、不能只有兩個，見下方「為什麼單筆查詢的路徑要帶完整鍵值」)。

這題也第一次用到 HTTP 狀態碼的 **4xx 家族**：`404 Not Found` 代表「呼叫端指定的資源不存在」。後面 rs05 / rs06 會陸續用到 `400` / `409`，整個 4xx 家族的共同語意是「這是呼叫端的問題，不是伺服器的問題」 (對照 5xx 家族代表「伺服器內部出錯」)。

學習目標

- 理解 `CL_REST_ROUTER` 的 `ATTACH` 樣板可以帶 {變數}，router 比對到請求 URL 後會把對應片段抽出來，交給 Resource Class 用 `MO_REQUEST->GET_URI_ATTRIBUTE( '變數名稱' )` 取值
- 分辨「集合查詢」 (`/flights`，回傳陣列) 跟「單筆查詢」 (`/flights/{carrid}/{connid}/{fldate}`，回傳單一物件) 是兩個不同的 **Resource Class**，各自對應一個 URI 樣板，不要塞進同一個 GET 方法裡用 if/else 判斷
- 理解為什麼單筆查詢的 URI 要帶**完整主鍵**：`SFLIGHT` 的主鍵是 `MANDT + CARRID + CONNID + FLDATE`，如果 URI 只給 `carrid` / `connid` 兩個欄位，`SELECT SINGLE` 抓到的會是「剛好排在最前面的那筆」，不是「使用者真正想要的那一筆」——這跟 rs02 用 `\t` 分欄純文字類似，是「看起來可以動，但語意是錯的」的坑
- 會在資源不存在時 `RAISE EXCEPTION TYPE cx_rest_resource_exception` 搭配 `status_code = cl_rest_status_code=>gc_client_error_not_found`，讓框架自動轉成 HTTP 404 (不用自己組錯誤 JSON，這部分框架已經處理，見 `CL_REST_RESOURCE~IF_REST_HANDLER~HANDLE` 裡的 `CATCH cx_rest_exception`)

為什麼單筆查詢的路徑要帶完整鍵值

如果沿用課綱草案原本設想的 `/flights/{carrid}/{connid}` (只有兩個路徑參數)，`SELECT SINGLE ... WHERE carrid = @lv_carrid AND connid = @lv_connid` 對 `SFLIGHT` 來說**不是唯一鍵值查詢**——同一個航班代號 + 航線代號，不同日期會有很多筆 (`AA / 0017` 可能有一百多個航班日期)。`SELECT SINGLE` 沒有 `ORDER BY` 時抓到的是資料庫剛好回傳的第一筆，換一次執行、換一台資料庫伺服器都可能不一樣，這種「表面上會動，但每次呼叫結果可能不同」的 API 是嚴重的設計缺陷。

正確做法：URI 要能唯一定位到一筆資料，路徑參數要涵蓋完整主鍵。這題把路徑改成 `/flights/{carrid}/{connid}/{fldate}`，三個欄位合起來剛好對應 `SFLIGHT` 主鍵 (扣掉系統自動處理的

MANDT) , SELECT SINGLE 才是真正「唯一」的查詢。

事前準備

ADT 端物件已由課程準備好 (\$TMP) :

- ZCL_RS04_APP——Application Class , router 掛兩個資源：集合 /flights 、單筆 /flights/{carrid}/{connid}/{fldate}
- ZCL_RS04_FLIGHTS——集合查詢，邏輯同 rs03 (20 筆 SFLIGHT 轉 JSON)
- ZCL_RS04_FLIGHT——單筆查詢，用完整主鍵查一筆 SFLIGHT , 查不到回 404

題目需求 (對照已建好的答案物件)

1. ZCL_RS04_APP 的 GET_ROOT_HANDLER : 一個 router 掛兩個 ATTACH

```
``abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template =
'/flights' iv_handler_class = 'ZCL_RS04_FLIGHTS' ). lo_router->attach( iv_template =
'/flights/{carrid}/{connid}/{fldate}' iv_handler_class = 'ZCL_RS04_FLIGHT' ).
ro_root_handler = lo_router. ``
```

1. ZCL_RS04_FLIGHTS : 跟 ZCL_RS03_FLIGHTS 完全相同的邏輯，這題重新建一份是因為每題的答案物件要能獨立閱讀，不用跳回前一題找程式碼
2. ZCL_RS04_FLIGHT 的 GET :

```
``abap DATA(lv_carrid) = mo_request->get_uri_attribute( 'carrid' ). DATA(lv_connid) = mo_request-
>get_uri_attribute( 'connid' ). DATA(lv_fldate) = CONV dats( mo_request->get_uri_attribute( 'fldate' ) ).
```

```
SELECT SINGLE carrid, connid, fldate, price, currency FROM sflight WHERE carrid = @lv_carrid AND connid =
@lv_connid AND fldate = @lv_fldate INTO @DATA(ls_flight).
```

```
IF sy-subrc <> 0. RAISE EXCEPTION TYPE cx_rest_resource_exception EXPORTING status_code =
cl_rest_status_code=>gc_client_error_not_found request_method = if_rest_message=>gc_method_get textid =
cx_rest_resource_exception=>resource_not_found. ENDIF.
```

```
DATA(lv_json) = /ui2/cl_json=>serialize( data = ls_flight pretty_name = /ui2/cl_json=>pretty_mode-camel_case
).
```

```
DATA(lo_entity) = mo_response->create_entity( ). lo_entity->set_string_data( lv_json ). lo_entity-
>set_content_type( if_rest_media_type=>gc_appl_json ). `` GET_URI_ATTRIBUTE( 'fldate' ) 拿回來是 STRING
( 例如 '20260115' ) , CONV dats( ... ) 轉成 DATS 型別——8 碼數字字串轉 DATS 是 ABAP 內建轉換規則，不
需要額外的轉換常式
```

注意這個輸入格式跟 rs03 的輸出格式不對稱：/flights 集合查詢回傳的 JSON , fldate 是 /UI2/CL_JSON 序列化出來的 "2026-01-15" (帶短橫線的 ISO 格式 , DATS 型別固定這樣輸出 , 不受 PRETTY_NAME 影響) ; 但這裡 CONV dats(...) 要吃的是不帶短橫線的 8 碼數字 '20260115' 。實際測試時，如果直接把集合查詢回傳的 fldate 字串整個貼到 URL 路徑上，會因為多了兩個 - 導致轉換失敗——要先手動去掉短橫線。這是刻意留下的真實案例 (2026-07-12 實測踩到) , 對照思考題第 1 題一起看。

SICF 掛載步驟 (比照 rs01~rs03 , 只是換掉數值)

沿用既有的 `/sap/bc/zrest_training` 分類節點，這題只要在其底下再加一個子節點：

1. SICF 交易碼，在 `zrest_training` node 上右鍵 → **New Sub-Element**，Service Name 填 `rs04`
2. Handler List 掛 `ZCL_RS04_APP`
3. Activate Service
4. 用 `SPROX_HTTP_REQUEST`（或連得到內網時用瀏覽器）測兩種路徑：**URL 要填完整網址（含 `http://` 與主機:port）**，不能只填 `/sap/bc/...` 相對路徑——`SPROX_HTTP_REQUEST` 雖然是在 Application Server 上跑的 ABAP 程式、位於內網，但它執行的是一支真正的 HTTP request（呼叫 ICM 的 HTTP Port），不是程式內部呼叫，沒給 `host:port` 會找不到這個 API endpoint
 - 集合：`http://<主機>:<port>/sap/bc/zrest_training/rs04/flights?sap-client=130`
 - 單筆（先從集合結果挑一筆真實存在的 `carrid/connid/fldate`，`connid` 用 JSON 給的數字即可、`fldate` 記得去掉短橫線）：`http://<主機>:<port>/sap/bc/zrest_training/rs04/flights/AA/17/20260115?sap-client=130`
 - 刻意測一個不存在的組合（例如 `fldate` 亂填），確認回應是 `404`

預期輸出（範例）

單筆查詢命中（URL 路徑用 `/flights/AA/17/20260115`，輸入是不帶短橫線的 8 碼數字；輸出的 JSON 一樣經過 `/UI2/CL_JSON` 序列化，`connid` 變不帶引號的數字、`fldate` 變帶短橫線的 ISO 格式，跟輸入格式不對稱，見上一節提醒）：

```
{"carrid":"AA","connid":17,"fldate":"2026-01-15","price":422.94,"currency":"USD"}
```

單筆查詢查不到：HTTP 狀態碼 `404`，回應內容是框架預設的錯誤文字（`RESOURCE_NOT_FOUND` 對應的訊息文字），不是 JSON——統一把所有錯誤都轉成一致的 JSON 格式，要留到 `rs07` 才處理。

團隊實務備註

- `CL_REST_ROUTER` 比對 `ATTACH` 樣板是精確片段數優先：`/flights` 跟 `/flights/{carrid}/{connid}/{fldate}` 的片段數不同（1 段 vs 4 段），不會互相搶著比對，這也是為什麼可以放心讓同一個 router 同時掛集合與單筆兩種樣板
- `GET_URI_ATTRIBUTE`（單數）一次取一個變數值，回傳 `STRING`；如果要一次拿全部路徑變數，`IF_REST_REQUEST` 也有複數版 `GET_URI_ATTRIBUTES`（），回傳 `TIHTTPNPV`（name-value pair 表格）——這題變數少，用單數版本比較直覺
- `CX_REST_RESOURCE_EXCEPTION` 是 `CL_REST_RESOURCE~IF_REST_HANDLER~HANDLE` 統一 `CATCH` 的例外類別，`RAISE EXCEPTION` 後不用自己再處理 HTTP 狀態碼或組回應——這是框架幫你做的事，寫 Resource Class 時應該優先用這個機制，而不是自己 `mo_response->set_status( 404 )` 再手動組錯誤內容

思考題

1. 如果呼叫端輸入的 `fldate` 不是 8 碼數字（例如 `/flights/AA/0017/abc`），`CONV dats( 'abc' )` 會發生什麼事？（這題原本的提示是錯的——寫這題當下猜測 `CONV` 轉型失敗會丟出例外，但 `rs07` 實際掛 SICF 測試後才發現：`CONV dats( ... )` 對非數字字串不會拋出任何例外，`DATS` 底層是字元型別、不做內容檢查，不合法的字串會被原樣塞進去，後續查詢資料庫只會單純「查無資料」，回傳的狀態碼會是 `404`，不是想像中因為轉型例外沒被攔到而導致的系統錯誤。這題目前的程式碼完全沒有對 `fldate` 格式做防禦，`abc` 這種輸入會直接以 `404` 收場，跟「`carrid` 打錯」的 `404` 混在一起分不出來；`rs07` 才第一

次補上「先驗證字元組成與長度，不要依賴轉型失敗」的正確做法，見 rs07 md 的 `read_keys` 實作與說明)

2. 承學習目標第 3 點：如果 `SFLIGHT` 沒有 `FLDATE` 這個欄位 (假設主鍵只有 `CARRID + CONNID`)，單筆查詢的 URI 設計會怎麼變？這說明了 REST 資源的 URI 結構其實是在反映 DB 表格的哪個部分？
3. 這題的集合查詢 (`/flights`) 跟單筆查詢 (`/flights/{carrid}/{connid}/{fldate}`) 回傳的 JSON 物件形狀 (欄位) 完全一樣，都是 `carrid/connid/fldate/price/currency` 五個欄位——這是刻意設計還是巧合？如果不一樣會有什麼問題？

答案

見 `zcl_rs04_app.clas.abap`、`zcl_rs04_flights.clas.abap`、`zcl_rs04_flight.clas.abap` (SAP 端物件 `ZCL_RS04_APP` / `ZCL_RS04_FLIGHTS` / `ZCL_RS04_FLIGHT`)。SICF Service 路徑 `/sap/bc/zrest_training/rs04`。